

条款18 对于独占资源使用 `std::unique_ptr`

① 核心作用: 独享所有权 无法拷贝 只能转移

特点: a. 独占 一块内存, 只能有一个 `unique_ptr` 管理

b. 自动释放 离开作用域, 自动释放内存

c. 零额外开销 大小与裸指针一样, 几乎一样快

② 基础用法

```
#include <memory>
```

```
std::unique_ptr<int> p1(new int(10));
```

// 更推荐 C++14

```
auto p2 = std::make_unique<int>(20);
```

```
*p2 = 100; // 解引用
```

```
p2.get(); // 获取裸指针 p2.reset(); // 主动释放内存
```

```
p2.reset(new int(20)); // 释放旧内存, 管理新内存
```

```
auto pp = std::move(p); // 转移所有权
```

③ 作函数返回值

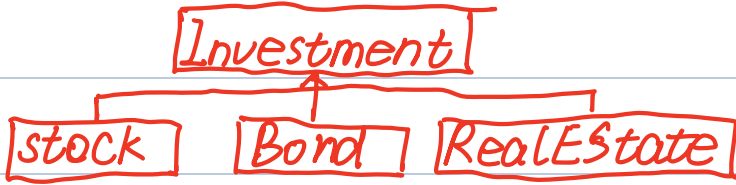
```
std::unique_ptr<int> f() {
```

```
    return std::make_unique<int>(20); }
```

```
auto p = f();
```

符合工厂函数 资源管理模式

④ 作为继承层次结构中对象的工厂函数返回类型



a. 为什么要用指针 → 多态

5. 为什么要用智能指针 → 栈上内存不好传递, 堆上内存不好控制

⑤ 智能指针的自定义删除器

$auto delInvmt1 = [](Investment* pInvestment) \{$

```
std::cout << "delete" << std::endl;
```

delete pInvestment[i];

```
void delInvestment2(Investment * pInvestment){
```

```
std::cout << "delete" << std::endl;
```

delete pInvestment[i];

```
std::unique_ptr<Investment> void(*) (Investment*)>
```

16字节 $\leftarrow$ `uptr2(NULLptr, delInvmt2);`

..... decltype(delInvmt1) uptr ... delInvmts ..

↓
8字节

具有很多状态的自定义删除器会产生较大尺寸的 `std::unique_ptr`

对象,如果发现自定义删除器使得`std::unique_ptr`变得过大,

则需要审视你的设计